

Taller de programación 2025

Unidad 3: Programación concurrente

Secuencial vs. concurrente

Si uno busca “concurrentia” en Google, podrá notar que esta palabra tiene varios significados:

1. *Acción y efecto de concurrir.*
2. *Conjunto de personas que asisten a un acto o reunión.*
3. *Coincidencia, concurso simultáneo de varias circunstancias.*
4. *Asistencia, participación.*

Pero como los informáticos no solemos participar de ningún acto o reunión, nos interesa únicamente el tercer significado mencionado. Por eso, en informática, *concurrentia* refiere a la habilidad que tiene un sistema para ejecutar múltiples tareas mediante la ejecución simultánea o de tiempo compartido.

Esto es algo nuevo. Hasta ahora habíamos trabajado siempre con la “programación secuencial”, donde cada tarea se completa antes de pasar a la siguiente. Ahora no: ahora vamos a tener varias tareas ejecutándose al mismo tiempo, ya sea de forma simultánea en diferentes núcleos de procesamiento (en paralelo), o de a ratitos, compartiendo los recursos del sistema (a veces llamado *time-sharing* o *context switching*).

¿Qué serían esas tareas/procesos? Eso depende del sistema. En el contexto de un sistema operativo, por ejemplo, serían los procesos involucrados en dicho sistema. En cambio, en el contexto de un sistema bancario (concretamente, en el software para cajeros automáticos), las tareas serían las diversas operaciones que puede realizar ese programa, tipo depositar o extraer dinero, consultar el saldo, etc.

En esta materia, obviamente, como vamos a estudiar la *programación concurrente*, vamos a trabajar con el concepto de concurrencia en el ámbito de los programas. Más específicamente, como vamos a trabajar con R-info, los procesos serán los robots.

Entonces, para dejar claras las cosas, acá va la definición:

*Un **programa concurrente** se divide en tareas (2 o más), las cuales se ejecutan al mismo tiempo y realizan acciones para cumplir un objetivo común. Para esto pueden: compartir recursos, coordinarse y cooperar.*

Esto tiene muchas ventajas para los usuarios, pero a la hora de programar la cosa se complica: ahora tendremos varios recursos compartidos (por ejemplo, ciertas esquinas), que deben manipularse con cuidado para evitar que se corrompa el sistema. Y para eso, es fundamental la coordinación y la cooperación. Hay que trabajar en conjunto, pero evitando meter mano donde no corresponde. Por eso, la comunicación es clave.

Messenger robótico

En el universo de la concurrencia, uno puede comunicarse mediante la memoria compartida o mediante mensajes. Con R-Info, la memoria compartida será el área de la ciudad compartida, y la comunicación mediante mensajes, será cuando enviamos y recibimos mensajes.



El pasaje de mensajes involucra dos procesos: el envío de mensajes, y la recepción de mensajes.

A su vez, la comunicación mediante mensajes puede ser asincrónica o sincrónica. Asincrónica significa que el proceso que envía/recibe el mensaje no espera que se de la comunicación para continuar su ejecución. Sincrónica, en cambio, que el proceso que envía/recibe el mensaje sí espera que se de la comunicación para continuar su ejecución.

En R-info, el envío de mensajes es asincrónico (es decir, el robot que envía el mensaje lo envía y sigue con su vida (ejecutando el resto de sus instrucciones) sin esperar que el robot receptor lo reciba), **y la recepción de mensajes es sincrónica** (es decir, el robot que espera un mensaje no hace nada (no ejecuta otras instrucciones) hasta que efectivamente recibe el mensaje).

Sobre el envío

Para enviar un mensaje, la instrucción es:

`EnviarMensaje(valor/variable, variableRobotDestinatario)`

donde el primer parámetro puede ser un valor inmediato o una variable de tipo número o boolean, y el segundo parámetro es el nombre de algún robot al cual le quiero enviar el mensaje. El nombre, no el tipo.

Solo puede haber un mensaje por cada envío.

Para que mi programa sea correcto, para cada envío que hay en mi programa debe haber una sentencia de recepción. Si esto no se cumple, es posible que el programa nunca finalice.

Sobre la recepción

Para recibir un mensaje, la instrucción es

`RecibirMensaje(valor/variable, variableRobotEmisor)`

Otro detalle: cuando estamos esperando recibir varios mensajes que pueden provenir de distintos robots y no sabemos cuál va a llegar primero, podemos escribir esto:

`RecibirMensaje (valor, *)`

lo cual significa que el `RecibirMensaje()` queda atento al envío de cualquier robot, y entonces atiende al primer mensaje que le llegue.

Ahora bien, esto tiene una desventaja: ganamos tiempo, pero perdimos información, porque ¿quién envió el mensaje? No lo sabemos: esa información no está en el asterisco.

Para solucionar este problema, tendremos que agregar un mensaje extra: el mensaje con el número de identificación del robot.

Veámoslo con un ejemplo.

Supongamos que tenemos un robot jefe y dos robots juntadores. El jefe primero le envía las instrucciones de dónde tienen que juntar las flores (le envía las dimensiones de un rectángulo), y se queda esperando a que ambos le digan cuánto juntó cada uno.

Como no sabemos quién va a terminar primero, usamos el asterisco para recibir la información (y lo usamos dos veces, ya que vamos a recibir dos mensajes). Hasta ahora, todo bien: vamos a recibir las dos cantidades.

Pero supongamos que queremos saber quién juntó más flores. ¿Cómo sabemos quién envió cada cantidad? Si no lo aclaramos, no lo sabemos: para esto está el mensaje con el id del robot, que será recibido en la variable *quien*. Así que ahora, cada juntador, además de enviar cuánto juntó, enviará su “DNI” así sabemos quién es.

En código (escribiendo únicamente lo esencial), sería algo así:

...

robots

robot juntador

variables

altura, ancho, f, quien : numero

comenzar

```
RecibirMensaje(quien,robot3) {el juntador recibe su id}
RecibirMensaje(altura,robot3) {recibe altura del rectángulo}
RecibirMensaje(ancho,robot3) {recibe ancho del rectángulo}
rectángulo (altura,ancho,f) {el juntador recorre y junta}
EnviarMensaje (quien,robot3) {le envía a su jefe su id}
EnviarMensaje (f,robot3) {le envía cuántas flores juntó}
```

fin

robot jefe

variables

max, rmax, f : numero

comenzar

```
max := 0
EnviarMensaje (1,robot1) {1° envía el id al robot1}
EnviarMensaje (5,robot1) {2° envía la altura al robot1}
EnviarMensaje (3,robot1) {3° envía el ancho al robot1}
EnviarMensaje (2,robot2) {4° envía el id al robot2}
EnviarMensaje (8,robot2) {5° envía la altura al robot2}
EnviarMensaje (2,robot2) {6° envía el ancho al robot2}
```

```

Repetir 2 {repite 2 porque va recibir 2 mensajes de 2 robots}
  RecibirMensaje(quien,*) {con *, pues no sabe quién enviará 1º}
  Si(quien = 1)
    RecibirMensaje(f,robot1)
  Sino
    RecibirMensaje(f,robot2)
  Si (f >= max)
    max := f
    rmax := quien
  Informar (rmax)
fin
fin
...

```

¡Liberen la esquina!

Como mencionamos anteriormente, otra manera de comunicarse es mediante la memoria compartida. Acá esa “memoria compartida” vienen a ser las esquinas compartidas.

¿Qué tiene de loco esto? Que si no coordinamos bien este recurso (las esquinas compartidas), los robots podrían chocar y morir en el intento de traficar sus flores y sus papeles.

Para que eso no suceda, los robots no tienen que estar al mismo tiempo en la misma esquina. Y para lograrlo, tenemos las palabras mágicas:

```

BloquearEsquina(av, ca)
...
LiberarEsquina(av, ca)

```

Importante:

- Una esquina DEBE ser bloqueada *si y sólo si* existe la posibilidad que dos o más robots puedan posicionarse en la misma al mismo momento.
- Las esquinas deben permanecer bloqueadas el menor tiempo posible: tiene que ser lo justo y necesario para que el robot haga lo que tenga que hacer ahí, pero ni un segundo más, así maximizamos la concurrencia.
- Una esquina bloqueada siempre debe ser desbloqueada en algún momento del programa. Al finalizar el programa NO pueden quedar esquinas bloqueadas.
- Lógicamente, el “protocolo” para acceder a una de estas esquinas compartidas es:
 - 1º La bloqueo
 - 2º Entro

- 3° Hago lo que tengo que hacer
- 4° Me voy a una esquina segura (= que no tenga riesgo de colisión)
- 5° La libero

FAQ

- ¿Que pasa si hay una esquina bloqueada por el robot1, y el robot2 también quiere bloquearla?

Solamente puede haber un robot a la vez en cada esquina. Por eso, si el robot1 la tiene bloqueada, el pedido de bloqueo del robot2 va a quedar en espera hasta que el robot1 la libere. Una vez desbloqueada, será bloqueada por el robot2. Esta lógica vale para x cantidad de robots queriendo bloquear una misma esquina.

Welcome to the desert of the real

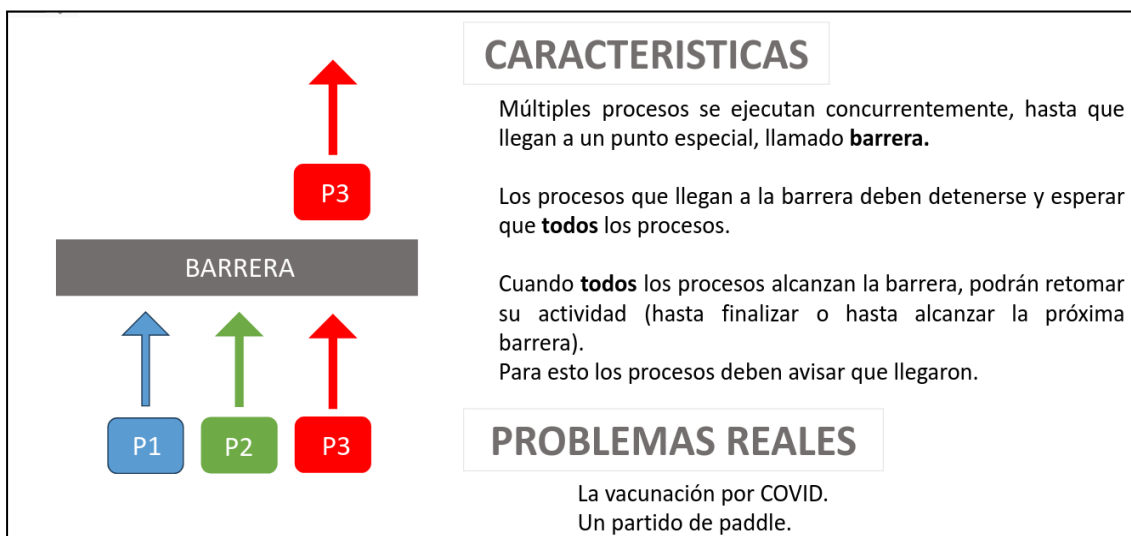
Hasta acá todo divino, programábamos a nuestro antojo. Pero en la vida real, hay ciertos problemas para los cuales ya existen algunas soluciones estandarizadas.

Algunos de los más importantes, son los siguientes cinco:

1) Sincronización por barrera

Algunos ejemplos concretos son: el 3 de la práctica 4.

Importante: debe crearse 1 tipo de robot para cada robot.



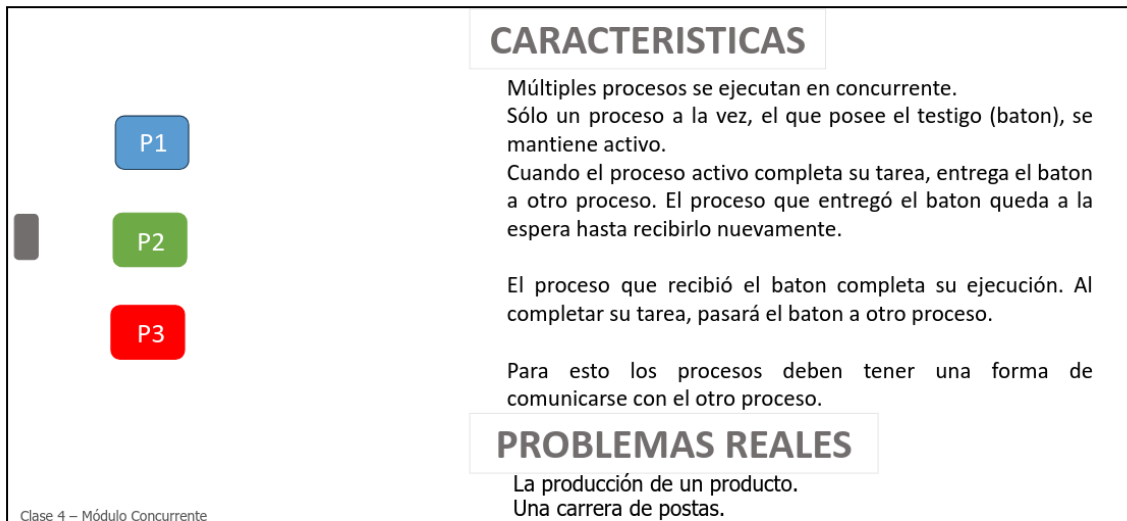
2) *Passing the baton*

Algunas traducciones posibles son: “pasar el testigo”, “pasar la batuta” y “pasar el bastón”.

Básicamente, la dinámica acá es como una carrera de postas.

Algunos ejemplos concretos son: el 3 de la práctica 2.

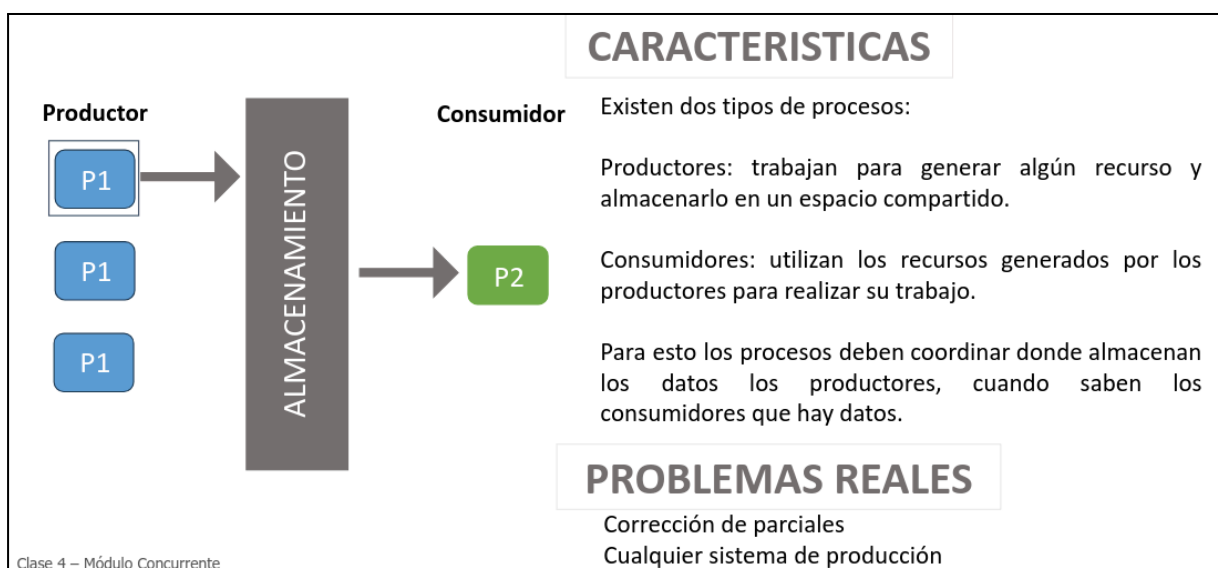
Importante: normalmente debe crearse 1 tipo de robot para cada robot (por razones similares a lo que sucede con la sincronización por barrera).



3) Productor/consumidor

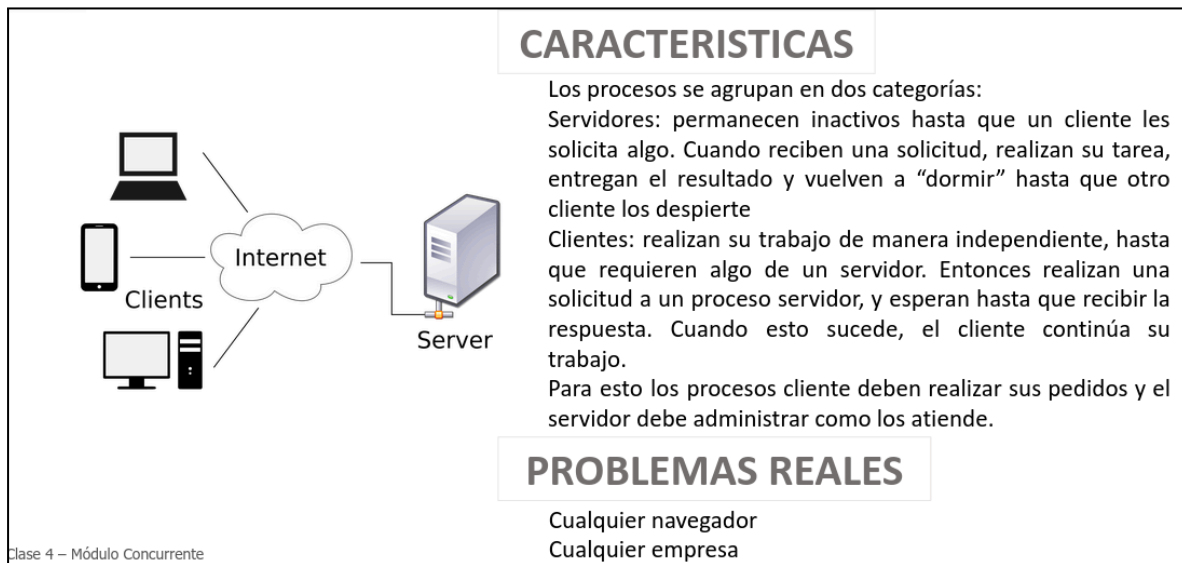
Uno o algunos produce(n), y otro(s) consume(n).

Algunos ejemplos concretos son: el 2 de la práctica 4.



4) Servidor/cliente

Algunos ejemplos concretos son: el 1 de la práctica 4 y el 5 de la práctica 5.



En la práctica 4 tenemos una descripción detallada de cómo sería este protocolo para un problema particular de R-Info:

Existe un robot que sirve de flores a tres robots clientes. Cada cliente solicita al servidor que le deposite en su esquina siguiente una cantidad de flores aleatoria (entre 1 y 4). Por ejemplo, si el cliente se encuentra en la esquina (2,1) le solicitará que coloque x cantidad de flores en la esquina (2,2).

Cuando el robot servidor deposita las flores en la esquina solicitada, el cliente las junta y las deposita una a una a lo largo de la avenida en la que se encuentra. El programa finaliza cuando todos los robots clientes completan su avenida. Asuma que el robot servidor tiene flores suficientes en su bolsa.

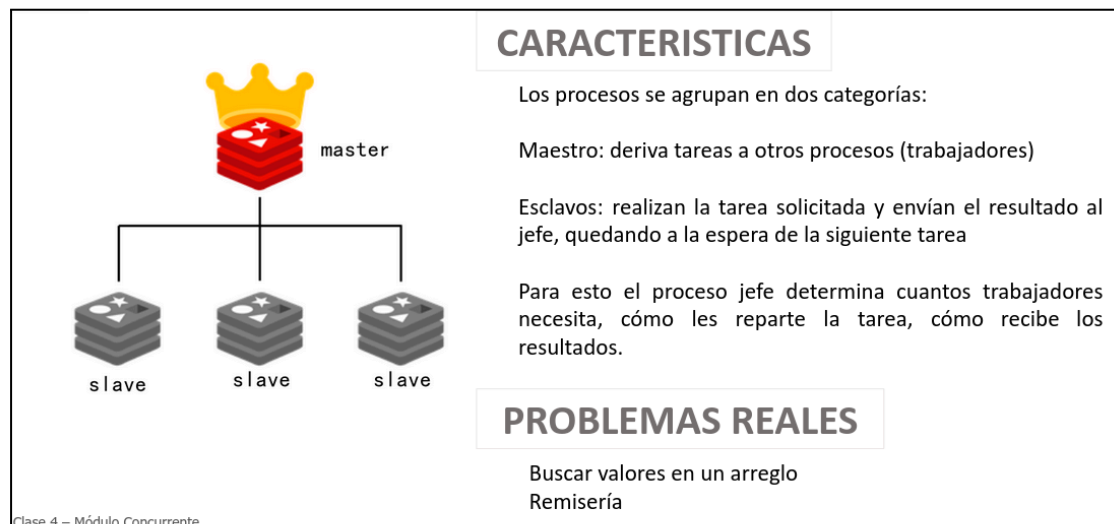
El robot servidor se inicia en la esquina (100, 100). Los robots clientes inician en las esquinas (1,1) , (2,1) y (3,1) respectivamente.

Protocolo cliente/servidor	
Cliente	Servidor
Recibir mi ID INICIO: calcularRandom flores Enviar ID al servidor Enviar cantFlores al servidor Enviar mi Avenida actual Enviar Calle siguiente Esperar ACK del servidor	Enviar ID a los clientes INICIO: Recibir ID Recibir N Flores de ID si (flores < 0) recibir avenida de ID recibir calle de ID pos(avenida, calle)

Ir a la esquina Avenida,Calle JuntarFlores Volver a la esquina Avanzar dejando flores Si llegué a la avenida 100 enviar 0 al servidor sino Volver a INICIO	depositar N flores volver a (100, 100) enviar ACK a robot ID volver a INICIO sino contar un robot terminado si terminaron los 3 robots terminar
Lo cual, en código, sería una cosa así:	
<pre> robot cliente variables ack, falta : boolean id, nFlores, miAv, miCa : numero comenzar miAv := PosAv falta := V RecibirMensaje(id, serv) mientras (falta = V) miCa := PosCa + 1 Random(nFlores, 1, 4) EnviarMensaje(id, serv) EnviarMensaje(nFlores, serv) EnviarMensaje(miAv, serv) EnviarMensaje(miCa, serv) RecibirMensaje(ack, serv) realizarTrabajo(nFlores, falta) EnviarMensaje(id, serv) EnviarMensaje(0, serv) fin </pre>	<pre> robot servidor variables id, cantRT, nFlores : numero comenzar cantRT := 0 enviarIdentificadores mientras (cantRT < 3) recibirIDandNUMFLO(id, nFlores) si (nFlores > 0) responderPedido(id, nFlores) sino cantRT := cantRT + 1 fin </pre>

5) Master/slave

Maestro/esclavo. Jefe/trabajador. Como prefieras. Hicimos muchos de este estilo.



Tips para que el programa no crashee y funcione *very nicely*

- Enviar y recibir mensajes desde el “programa principal”, no desde los procesos.
- No usar nombres de variables súper cortos como `cF`. Mejor algo como `cantFlo`.
- No poner la condición con boolean sola. Me refiero a esto: no escribir una condición de esta manera: `si (ok) ...`, mejor escribirla explícitamente, así: `si (ok = V)...`
- Ninguna línea de código debe ocupar todo el ancho de la pantalla, ya que si se sobrepasa, seguramente funcione mal.
- Cuidado con el contenido de `EnviarMensaje()` y `RecibirMensaje()`. Hay atajos que suelen funcionar mal, como poner `EnviarMensaje(PosAv, r1)`. En vez de eso, habría que tomar el valor de `PosAv` en una variable auxiliar de tipo número, y recién ahí mandarlo.
- La definición de las áreas, en términos de privacidad/publicidad, debe ser exacta.
 - 1) Si yo pongo un área privada (P), entonces sí o sí esa área debe estar asignada a 1 robot, ni más ni menos que a uno.
 - 2) Si pongo un área parcialmente compartida (PC), esa área debe estar asignada a 2 o más robots, y debe existir al menos uno que no la tenga asignada.
 - 3) Si es compartida (C), todos los robots tienen que tener esa área asignada.

Parece una pavada, pero si uno hace mal esto es probable que al finalizar el programa no nos aparezca el cartelito “Programa finalizado”, y uno se queda buscando el error en cualquier lado, y piensa que faltó recibir un mensaje o algo así.

- Para que R-Info ande bien, suele convenir tener una versión relativamente nueva de Java.
- No superponer áreas. O sea, por más que R-Info te lo permita, está mal conceptualmente, y puede que haga funcionar mal el programa.

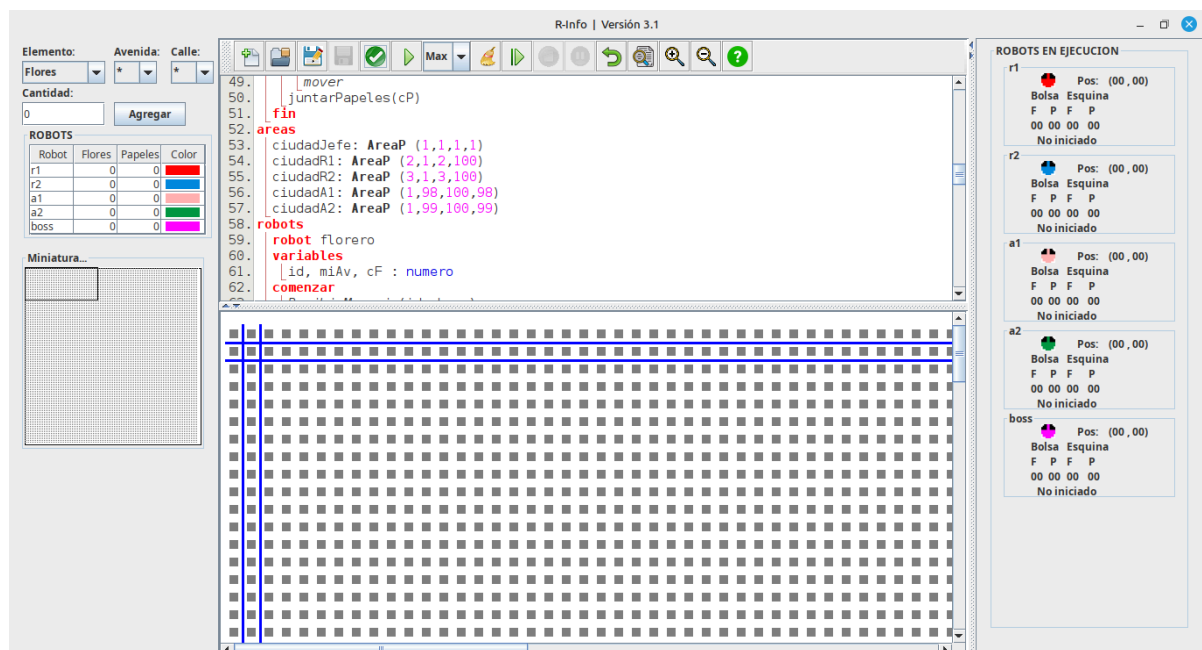
Ejemplo concreto: el ejercicio 1 de la práctica 5 dice:

Se organizó una competencia entre el equipo rojo y el equipo azul. Cada equipo consta de dos robots, y debe realizar una tarea:

- *Los robots R1 y R2 del equipo rojo debe juntar todas las flores de las avenidas 2 y 3 respectivamente.*
- *Los robots A1 y A2 del equipo azul debe juntar todos los papeles de las calles 98 y 99 respectivamente.*

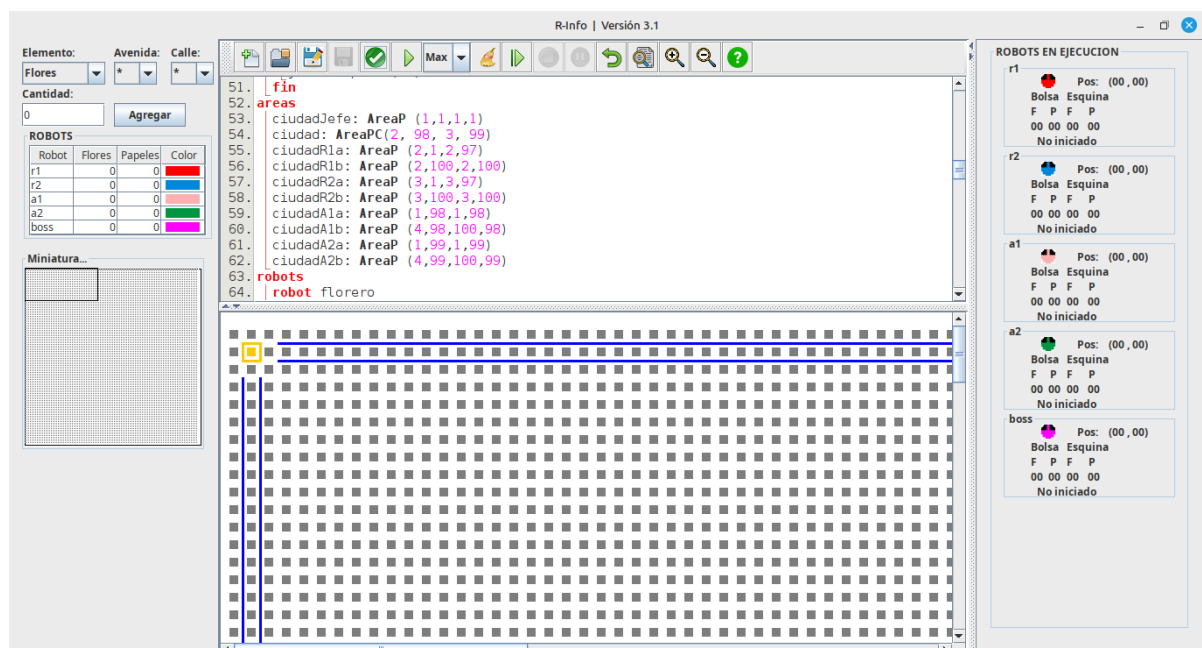
Al finalizar la competencia, un robot fiscalizador deberá informar el equipo que juntó más objetos.

Podríamos hacer algo como esto: una sola área privada para cada calle y avenida.



El programa funcionaría, pero sería incorrecto, porque el área donde se cruzan las áreas privadas (es decir, las esquinas donde podrían colisionar los robots), no son áreas privadas, sino áreas compartidas o parcialmente compartidas (esto depende del ejercicio, en este caso sería parcialmente compartida ya que hay al menos un robot que nunca iría a esa área (a saber, el robot jefe)).

Entonces, lo correcto sería esto:



Tips para programar en R-Info Like a Pro 100% Real No Fake

1. "Todo lo que pueda ser privado, será privado". O sea, si un robot tiene que recorrer él solo algo, lo ideal es que lo haga en un área privada.

2. "Cuanto más ajustada sea el área, mejor". Es decir, hay que delimitar el área lo mejor posible, para abarcar únicamente el espacio que el robot necesita, sin que quede espacio vacío.

Ejemplo que aplica estas dos máximas: si tengo que hacer un robot que recorra desde (1, 1) hasta (1, 2), entonces todo lo que necesito es esta ciudad: AreaP (1, 1, 1, 2). O sea, un área privada con la extensión que va a usa el robot. Nada más.

3. "Cuanto menos tipos de robots, mejor". Obviamente, la idea es reutilizar código lo más que podamos. Si tenemos dos robots que hacen lo mismo, pero empiezan en lugares distintos, seguramente podamos usar el mismo tipo de robot para ambos.

Pero ojo, no te pases de listo. Cuando dos robots hacen *casi* lo mismo (como en el ejercicio 1 de la práctica 2), tenemos que evitar este tipo de truquitos:

```
robot tipo1
variables
  cF, cFotro : numero
comenzar
  juntarLinea(9, cF)
  si (PosAv = 1)
    EnviarMensaje(cF, robot2)
  sino
    EnviarMensaje(cF, robot1)
  RecibirMensaje(cFotro, *)
  si (cF > cFotro)
    Informar("La diferencia de flores es ", cF - cFotro)
fin
```

Así que, **si los robots no hacen lo mismo, hay que hacer distintos tipos de robots.** Particularmente, si implementamos la sincronización por barrera, debemos hacer un tipo de robot para cada robot, ya que cada robot no envía ni recibe los mensajes hacia/desde los mismos robots. *Idem* para *Passing the baton*.

4. ¡Se puede imprimir “Hola mundo” en pantalla! Es así:

```
Informar("Hello Bro!", variable)
```

El texto va entre comillas, después una coma, y después una variable o un valor inmediato. Sí o sí es con ese formato. No se puede imprimir puro texto. Para hacer eso, la convención es poner el texto y después el valor inmediato 0.

Eso sí, ojo con lo que ponés entre comillas: R-Info no acepta casi ningún carácter especial ni palabra clave. Si hay algo tipo el signo : (dos puntos) o la palabra *robot*, va a fallar. Así que mejor poné letras y palabras que no formen parte del código.

5 - Si hay un jefe y varios robots, y el jefe tiene que poder identificarlos, el jefe debe ser el encargado de enviarle la ID a cada uno. *No podemos asignarle la ID a cada robot haciendo hardcoding.*

Ejemplo: en el ejercicio 2 de la práctica 3 hay dos tipos de robots y solo un ejemplar de cada tipo de robot, por lo tanto, podríamos asignarle en el propio código de cada robot su propia ID. La ventaja sería que cada robot no tiene que esperar a recibir el ID desde el jefe. Es una solución más eficiente, pero ESTÁ MAL, porque no es escalable.

6 - Si el jefe tiene que informar un ganador (siendo este el valor máximo recibido de alguno de los robots), no hace falta que el jefe almacene todos los valores: puede ir calculando el máximo a medida que los recibe.

O sea, en vez de hacer todo esto:

```
proceso recibirCants(ES c1 : numero; ES c2 : numero; ...)
variables
  id : numero
comenzar
  repetir 4
    RecibirMensaje(id, *)
    si (id = 1)
      RecibirMensaje(c1, r1)
    sino
      si (id = 2)
        RecibirMensaje(c2, r2)
      sino
        si (id = 3)
          RecibirMensaje(c3, r3)
        sino
          si (id = 4)
            RecibirMensaje(c4, r4)
fin
```

```

proceso calcGan(E c1 : numero; E c2 : numero; ... ; ES id : numero)
variables
    max : numero
comenzar
    si (c1 > c2)
        max := c1
        id := 1
    sino
        max := c2
        id := 2
    si (c3 > max)
        max := c3
        id := 3
    si (c4 > max)
        max := c4
        id := 4
fin

```

Habría que hacer esto: recibir todos los mensajes en el programa principal e ir actualizando el máximo.

```

proceso actualizarMax(E cant : numero; ES maxCANT : numero; E id :
numero; ES maxID : numero)
comenzar
    si (cant > maxCANT)
        maxCANT := cant
        maxID := id
fin

```

... {parte anterior del programa principal}

```

maxCANT := -1
repetir 4
    RecibirMensaje(id, *)
    si (id = 1)
        RecibirMensaje(cant, r1)
    sino
        si (id = 2)
            RecibirMensaje(cant, r2)
        sino
            si (id = 3)
                RecibirMensaje(cant, r3)
            sino
                RecibirMensaje(cant, r4)
        actualizarMax(cant, maxCANT, id, maxID)

```

... {lo que siga en el programa principal}

7 - Siempre que sea posible, hay que intentar maximizar la concurrencia.

O sea, siempre que podamos hacer que muchos robots laburen en simultáneo, o haciendo que estén activos lo más seguido y el mayor tiempo posible, mejor.

Esto se evidencia con el tema de bloquear y liberar esquinas. Si yo tengo que hacer algo en una esquina bloqueada, hago lo que tengo que hacer, y listo, me voy y la libero. No hay que ocupar esquinas al pedo: sería como ir a un baño público y quedarse 1 hora mirando TicToc. No flaco, el inodoro es para depositar desechos tóxicos, no para usar el celu. Vas al baño, dejás lo tuyo, y te vas. Punto.

Por otro lado, hay que ser solidario con los demás y no acaparar los recursos. Si tengo que depositar flores en una esquina compartida (y hay otros que van a querer hacer lo mismo), siempre que pueda conviene depositarlas de a una.

¿Por qué? Supongamos que dos robots (r1 y r2) quieren depositar sus flores en (50, 50). El robot r1 tiene 1.000.000 de flores y el r2 tiene solamente 2 florcitas. Si r1 le gana el pedido de bloquear esquina a r2, r1 la va a ocupar mucho tiempo (okey, sí, mucho tiempo haciendo un trabajo necesario, pero mucho tiempo al fin), y r2 se va a quedar esperando un montón, cuando en realidad podría haber tardado poco y nada si se la hubieran prestado a él primero dos segunditos, y ya hubiera quedado listo para hacer otra cosa. En cambio, si se comparten un poco el recurso, en intervalos cortos, eso no pasaría: el que tiene poco terminaría relativamente rápido, independientemente de que le haya tocado primero o no.

¿Solución? En vez de escribir esto:

```
BloquearEsquina(50, 50)
Pos(50, 50)
repetir cantFlores
    depositarFlor
Pos(miAv, miCa)
LiberarEsquina(50, 50)
```

Escribimos esto:

```
repetir cantFlores
    BloquearEsquina(50, 50)
    Pos(50, 50)
    depositarFlor
    Pos(miAv, miCa)
    LiberarEsquina(50, 50)
```

Obviamente el enunciado no siempre lo permite. Pero cuando se pueda, metele que sea así: *one by one*.